

The Ultimate ANOVA & Experimental Design Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	4
2	Part 0 — The 30-Second Exam Strategy	5
3	Part 1 — Foundations of Experimental Design	5
3.1	1.1 What is an experiment?	5
3.2	1.2 The three pillars of experimental design (Fisher)	5
3.3	1.3 Randomization — why?	5
3.4	1.4 Replication — why?	6
3.5	1.5 Blocking — why?	6
3.6	1.6 Confounding and aliasing	6
3.7	1.7 Control vs treatment	7
3.8	1.8 Fixed vs random effects (preview)	7
3.9	1.9 Observational vs experimental	7
3.10	1.10 The Experimental Design Process	7
4	Part 2 — Key Terminology	7
4.1	2.1 Design vocabulary	7
4.2	2.2 Model vocabulary	8
4.3	2.3 Design types at a glance	8
4.4	2.4 Notation	8
5	Part 3 — Randomization, Replication, Blocking	8
5.1	3.1 Randomization in R	8
5.2	3.2 Replication and pseudo-replication	9
5.3	3.3 Blocking decisions	10
5.4	3.4 Randomization restrictions	10
5.5	3.5 Common randomization mistakes	10
6	Part 4 — CRD: Completely Randomized Design	10
6.1	4.1 What it is	10
6.2	4.2 Generate the design	10
6.3	4.3 Fit and report	11
6.4	4.4 Post-hoc	12
6.5	4.5 CRD summary	12
7	Part 5 — RCBD: Randomized Complete Block Design	12
7.1	5.1 What it is	12
7.2	5.2 Design and data	12
7.3	5.3 Fit and report	13
7.4	5.4 Testing for interaction	13

7.5	5.5 When to use RCBD	14
7.6	5.6 RCBD summary	14
8	Part 6 — Latin Square Design	14
8.1	6.1 What it is	14
8.2	6.2 Generate and analyze	15
9	Part 7 — BIBD: Balanced Incomplete Block Design	16
9.1	7.1 What it is	16
9.2	7.2 Example	17
9.3	7.3 Analysis	17
9.4	7.4 BIBD summary	18
10	Part 8 — Full Factorial Designs	18
10.1	8.1 What it is	18
10.2	8.2 2^k factorial	18
10.3	8.3 2^3 with blocking	19
10.4	8.4 Interaction plots	20
10.5	8.5 Factorial design summary	20
10.6	8.6 When to use	21
11	Part 9 — Fractional Factorial Designs	21
11.1	9.1 What it is	21
11.2	9.2 Resolution	21
11.3	9.3 Generate the design	21
11.4	9.6 Fractional factorial summary	23
11.5	9.7 Blocking in factorial designs	23
12	Part 10 — Split-Plot Designs	24
12.1	10.1 What it is	24
12.2	10.2 Two error terms	24
12.3	10.3 Generate and analyze	24
12.4	10.4 Split-plot with lmer	25
12.5	10.5 Split-plot summary	26
12.6	10.6 Split-split-plot and strip-plot	26
13	Part 11 — Nested vs Crossed Designs	26
13.1	11.1 Crossed design	26
13.2	11.2 Nested design	26
13.3	11.3 Example	26
13.4	11.4 Nested vs crossed summary	27
14	Part 12 — Crossover Designs	27
14.1	12.1 What it is	27
14.2	12.2 2×2 crossover	27
14.3	12.3 Crossover summary	28
15	Part 13 — Repeated Measures ANOVA	28
15.1	13.1 What it is	28
15.2	13.2 Fit and report	28
15.3	13.3 Reading the output	29
15.4	13.4 Sphericity and corrections	29
15.5	13.5 lmer alternative	30
16	Part 14 — Mixed Designs (Between + Within)	30

16.1	14.1	What it is	30
16.2	14.2	Fit and report	30
16.3	14.3	Reading the output	31
16.4	14.4	Mixed-design summary	31
17	Part 15	Fixed vs Random Effects	32
17.1	15.1	The distinction	32
17.2	15.2	Why it matters	32
17.3	15.3	Mixed models	32
17.4	15.4	Rule of thumb	33
17.5	15.5	Mixed-model terminology	33
18	Part 16	Expected Mean Squares (EMS)	33
18.1	16.1	Why EMS matters	33
18.2	16.2	EMS table for common designs	33
	18.2.1	One-way ANOVA (fixed treatment)	33
	18.2.2	One-way ANOVA (random treatment)	34
	18.2.3	Two-way factorial (A, B fixed)	34
	18.2.4	Two-way factorial (A fixed, B random)	34
	18.2.5	Split-plot (A whole-plot fixed, B split-plot fixed)	34
18.3	16.3	EMS summary	35
19	Part 17	Power and Sample Size	35
19.1	17.1	Key parameters	35
19.2	17.2	Power for one-way ANOVA	35
19.3	17.3	Sample size needed	35
19.4	17.4	Power for factorial designs	36
19.5	17.5	Power for RCBD	36
19.6	17.6	Power for RM	36
19.7	17.7	Post-hoc power	36
19.8	17.8	Effect size benchmarks	37
20	Part 18	Blocking Efficiency	37
20.1	18.1	Why blocking helps	37
20.2	18.2	Efficiency formula	37
20.3	18.3	Computing in R	37
20.4	18.4	When to block	38
21	Part 19	Diagnostics	38
21.1	19.1	Assumptions	38
21.2	19.2	Checking in R	38
21.3	19.3	Interpreting the plots	40
21.4	19.4	What to do when assumptions fail	40
21.5	19.5	Diagnostic report template	40
22	Part 20	Post-hoc Comparisons	40
22.1	20.1	When to use which	40
22.2	20.2	Tukey HSD	40
22.3	20.3	Bonferroni	41
22.4	20.4	emmeans (recommended)	41
22.5	20.5	Games-Howell (unequal variances)	42
22.6	20.6	Simple effects for interactions	43
22.7	20.7	Reporting template	43
23	Part 21	Non-parametric Alternatives	43

23.1	21.1	One-way: Kruskal-Wallis	43
23.2	21.2	Repeated measures: Friedman	44
23.3	21.3	Factorial non-parametric: ART	45
23.4	21.4	Non-parametric summary	45
23.5	21.5	Reporting template	45
24 Part 22 — Theory Q&A			45
25 Part 23 — Exam Scenarios			47
25.1	23.1	You are given a table of data and asked to analyze	47
25.2	23.2	“Design an experiment to test X”	47
25.3	23.3	“Test if factor A affects the response”	47
25.4	23.4	“Compare 4 groups”	47
25.5	23.5	“Test interaction between A and B”	47
25.6	23.6	“Analyze a repeated measures experiment”	47
25.7	23.7	“Analyze a split-plot experiment”	48
25.8	23.8	“Design and analyze a factorial experiment”	48
25.9	23.9	“Blocking vs no blocking”	48
25.10	23.10	“Choose between two designs”	48
26 Part 24 — Quick Reference			48
27 Part 25 — Common Mistakes			49
28 Part 26 — The Single Most Important Idea			49

1 How to Use This Cheat Sheet

Topic	Section
The 30-second strategy	0
Foundations of experimental design	1
Key terminology	2
Randomization, replication, blocking	3
CRD — Completely Randomized Design	4
RCBD — Randomized Complete Block Design	5
Latin Square Design	6
BIBD — Balanced Incomplete Block	7
Full Factorial Designs	8
Fractional Factorial Designs	9
Split-Plot Designs	10
Nested vs Crossed Designs	11
Crossover Designs	12
Repeated Measures ANOVA	13
Mixed Designs (between + within)	14
Fixed vs Random Effects	15
Expected Mean Squares (EMS)	16
Power and Sample Size	17
Blocking Efficiency	18
Diagnostics	19
Post-hoc Comparisons	20
Non-parametric Alternatives	21
Theory Q&A	22
Exam Scenarios	23

Topic	Section
Quick Reference	24
Common Mistakes	25

Golden rule of this document: every chunk is self-contained. Each one starts with `set.seed()`, builds its own data, and uses section-prefixed names.

2 Part 0 — The 30-Second Exam Strategy

For **any** experimental design / ANOVA question, run this pipeline:

1. **Identify the design.** How many factors? Between or within? Blocks? Nested?
2. **Identify the model.** What are the fixed effects, random effects, and error terms?
3. **Fit** with the right function:
 - `aov()` for balanced fixed-effects
 - `lm()` for ANCOVA / unbalanced Type III
 - `aov(y ~ A + Error(block/A))` for split-plot
 - `lmer()` for mixed / unbalanced / missing
4. **Check assumptions:** Shapiro-Wilk, Levene, QQ plot.
5. **Read the ANOVA table.** Find F, df, p for the effect(s) of interest.
6. **If significant, post-hoc:** Tukey, Bonferroni, `emmeans`.
7. **Report effect size** and interpret in words.

If you can do these seven steps, you can answer any design question.

3 Part 1 — Foundations of Experimental Design

3.1 1.1 What is an experiment?

An **experiment** is a controlled study in which the researcher **manipulates** one or more factors (treatments) and observes the effect on one or more responses.

Key elements: - **Treatment:** a specific combination of factor levels applied to an experimental unit. - **Experimental unit:** the smallest unit to which a treatment is applied independently. - **Observational unit:** the unit on which the response is measured (may be smaller than the experimental unit). - **Response:** the measured outcome.

Why experiments beat observational studies: Randomization + control allows causal inference.

3.2 1.2 The three pillars of experimental design (Fisher)

1. **Randomization:** Assign treatments to units randomly to eliminate systematic bias.
2. **Replication:** Apply each treatment to multiple units to estimate variability.
3. **Blocking:** Group similar units together to reduce nuisance variation.

Mnemonic: Randomize, Replicate, Block.

3.3 1.3 Randomization — why?

Without randomization: - **Confounding:** the treatment effect is mixed with an unmeasured variable. - **Bias:** the researcher might unconsciously assign treatments non-randomly. - **Invalid p-values:** the F-test assumes observations are independent.

How to randomize in R:

```
# --- Self-contained: random assignment ---
set.seed(101)
p1_units <- 1:12
p1_treatments <- rep(c("A", "B", "C"), each = 4)
p1_design <- tibble(
  unit      = p1_units,
  treatment = sample(p1_treatments)
)
p1_design

## # A tibble: 12 x 2
##   unit treatment
##   <int> <chr>
## 1     1 C
## 2     2 C
## 3     3 B
## 4     4 A
## 5     5 A
## 6     6 C
## 7     7 B
## 8     8 A
## 9     9 A
## 10    10 B
## 11    11 C
## 12    12 B
```

3.4 1.4 Replication — why?

- **Estimates error variance.** Without replication, you cannot estimate the within-treatment variability.
- **Increases power.** More replication → smaller SE → easier to detect real effects.
- **Reduces chance of misleading results.** A single unit can be atypical.

Note: Replication = repeated measures. Repeating the measurement on the same unit does not give independent replication.

3.5 1.5 Blocking — why?

- **Reduces nuisance variability.** If a nuisance factor (e.g., batch, field, time of day) affects the response, block on it.
- **Increases precision.** By removing block-to-block variation from the error term, the residual SS shrinks, and the F-test for treatment becomes more powerful.

Rule: Block on the factor you can control, randomize what you cannot.

3.6 1.6 Confounding and aliasing

- **Confounding:** the effect of one factor is mixed with another (or a block). If they are always applied together, you cannot separate their effects.
- **Aliasing:** in fractional factorial designs, higher-order interactions are deliberately confounded with lower-order effects to reduce the number of runs.

Why it matters: Confounding destroys causal inference. Aliasing trades information for efficiency.

3.7 1.7 Control vs treatment

- **Control group:** no treatment / baseline (or the current standard).
- **Treatment groups:** different levels of the experimental factor.
- **Placebo control:** for clinical trials, a treatment that has no active ingredient.

3.8 1.8 Fixed vs random effects (preview)

Type	Meaning	Examples
Fixed	Levels chosen specifically by the researcher; inference only to those levels	Treatment, dose, sex
Random	Levels are a random sample from a larger population	Subject, batch, site, block

Implication: Fixed effects affect the mean; random effects affect the variance. The choice changes the EMS and the F-test denominator.

3.9 1.9 Observational vs experimental

Feature	Experiment	Observational
Treatment assignment	Random	Not controlled
Causal inference	Yes	No
Confounding	Controlled by design	Possible
Typical analysis	ANOVA	Regression / propensity score

3.10 1.10 The Experimental Design Process

1. **State the objective** — what question are you answering?
2. **Choose factors and levels** — what treatments?
3. **Choose the design** — CRD, RCBD, factorial, split-plot, ...
4. **Determine sample size** — power analysis.
5. **Randomize** — assign treatments to units.
6. **Run the experiment** — collect data.
7. **Analyze** — ANOVA + post-hoc.
8. **Interpret** — effects, sizes, practical significance.

4 Part 2 — Key Terminology

4.1 2.1 Design vocabulary

Term	Meaning
Factor	An independent variable manipulated by the researcher
Level	A specific value of a factor
Treatment	A combination of factor levels
Experimental unit	Unit to which a treatment is applied
Observational unit	Unit on which the response is measured
Replication	Independent application of a treatment

Term	Meaning
Block	Group of similar experimental units
Balance	Equal replication across treatments
Orthogonal	Effects are independent
Confounded	Effects cannot be separated
Nuisance factor	Affects response but not of primary interest

4.2 2.2 Model vocabulary

Term	Meaning
Fixed effect	Level-specific effect; inference to those levels only
Random effect	Drawn from a distribution; inference to the population
Mixed model	Contains both fixed and random effects
Error term	Residual variability not explained by the model
EMS	Expected Mean Square — used to determine F-test denominators
Residual	Observed — fitted

4.3 2.3 Design types at a glance

Design	Factors	Blocking	Typical use
CRD	1+ between	None	Simple comparison
RCBD	1 between + 1 block	Yes	One nuisance factor
Latin Square	1 between + 2 blocks	Yes (2-way)	Two nuisance factors
Factorial	2+ between	Optional	Interaction questions
Fractional Factorial	Many between	Yes (aliased)	Screening
Split-Plot	Between + within	Yes	Hard-to-change factors
Nested	Factor within factor	—	Hierarchical data
Crossover	Within-subject treatments	Yes (subject)	Clinical trials

4.4 2.4 Notation

- y_{ij} : response for unit i in treatment j .
- μ : grand mean.
- τ_j : effect of treatment j .
- β_i : effect of block i .
- $(\tau\beta)_{ij}$: interaction between treatment j and block i .
- ε_{ij} : random error, $\sim N(0, \sigma^2)$.

5 Part 3 — Randomization, Replication, Blocking

5.1 3.1 Randomization in R

Completely random assignment (CRD):


```
# --- Self-contained: CRD randomization ---
set.seed(301)
p3_treat <- rep(c("A", "B", "C"), each = 4)
p3_design <- tibble(
  unit      = 1:12,
  treatment = sample(p3_treat)
)
p3_design
```

```
## # A tibble: 12 x 2
##   unit treatment
##   <int> <chr>
## 1     1 C
## 2     2 C
## 3     3 B
## 4     4 C
## 5     5 C
## 6     6 B
## 7     7 A
## 8     8 A
## 9     9 A
## 10    10 B
## 11    11 A
## 12    12 B
```

Randomized block design (RCBD): randomize within each block.

```
# --- Self-contained: RCBD randomization ---
set.seed(302)
p3_design <- expand_grid(block = 1:4, treatment = c("A", "B", "C")) %>%
  group_by(block) %>%
  mutate(treatment = sample(treatment)) %>%
  ungroup() %>%
  arrange(block, treatment)
p3_design
```

```
## # A tibble: 12 x 2
##   block treatment
##   <int> <fct>
## 1     1 A
## 2     1 B
## 3     1 C
## 4     2 A
## 5     2 B
## 6     2 C
## 7     3 A
## 8     3 B
## 9     3 C
## 10    4 A
## 11    4 B
## 12    4 C
```

5.2 3.2 Replication and pseudo-replication

- **True replication:** independent units receiving the same treatment.

- **Pseudo-replication:** treating subsamples as replicates when they are not independent (e.g., measuring the same plant twice).

Pseudo-replication inflates Type I error. If you have subsamples, average them first or use a mixed model with a random effect for the unit.

5.3 3.3 Blocking decisions

Block when: - A nuisance factor is known and measurable. - That factor is likely to affect the response. - You can group units into homogeneous blocks.

Do not block when: - The nuisance factor is negligible. - Blocking uses up df that could be used for treatments.

Blocking efficiency: see Part 18.

5.4 3.4 Randomization restrictions

Design	Restriction
CRD	None (full randomization)
RCBD	Within-block randomization
Latin Square	Randomize rows and columns, then treatments
Split-plot	Randomize whole-plot factor, then split-plot factor within

5.5 3.5 Common randomization mistakes

1. **Assigning treatments by convenience** (e.g., first row gets treatment A) — biases results.
2. **Not randomizing block order** — confounds block with time.
3. **Using the same randomization for repeated experiments** — different experiments need different seeds.
4. **Forgetting to randomize measurement order** — measurement order can be a nuisance factor.
5. **Blocking after seeing the data** — post-hoc blocking is weaker than pre-planned blocking.

6 Part 4 — CRD: Completely Randomized Design

6.1 4.1 What it is

- One or more treatment factors.
- No blocking.
- Each experimental unit is randomly assigned to a treatment.
- Analysis: one-way ANOVA.

Model: $y_{ij} = \mu + \tau_j + \varepsilon_{ij}$

6.2 4.2 Generate the design

```
# --- Self-contained: CRD design and data ---
set.seed(401)
p4_design <- tibble(
  unit      = 1:20,
  treatment = factor(rep(c("A", "B", "C", "D"), each = 5),
```

```

        levels = c("A", "B", "C", "D")) %>%
    sample() # randomize assignment to units
)
p4_design$y <- 10 + c(A = 0, B = 2, C = 4, D = 6)[as.character(p4_design$treatment)] +
    rnorm(20, 0, 1.5)

p4_design

## # A tibble: 20 x 3
##   unit treatment      y
##   <int> <fct>      <dbl>
## 1     1 B      11.3
## 2     2 C      15.0
## 3     3 A      12.0
## 4     4 A       7.90
## 5     5 C      12.9
## 6     6 A      10.7
## 7     7 C      11.0
## 8     8 C      13.4
## 9     9 B      12.6
## 10    10 D      15.1
## 11    11 C      12.3
## 12    12 B      10.7
## 13    13 D      15.1
## 14    14 D      18.6
## 15    15 A      10.7
## 16    16 D      16.2
## 17    17 D      15.4
## 18    18 B      11.9
## 19    19 A      12.7
## 20    20 B      11.2

```

6.3 4.3 Fit and report

```

# --- Self-contained: CRD one-way ANOVA ---
set.seed(402)
p4_df <- tibble(
  treatment = factor(rep(c("A", "B", "C", "D"), each = 5),
    levels = c("A", "B", "C", "D")),
  y = 10 + c(A = 0, B = 2, C = 4, D = 6)[rep(1:4, each = 5)] + rnorm(20, 0, 1.5)
)
p4_fit <- aov(y ~ treatment, data = p4_df)
summary(p4_fit)

##           Df Sum Sq Mean Sq F value    Pr(>F)
## treatment   3  55.48   18.493    8.908 0.00105 **
## Residuals  16  33.22    2.076
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Report template: “A one-way ANOVA shows a significant effect of treatment, $F(3, 16) = 24.8$, $p < 0.001$.”

6.4 4.4 Post-hoc

```
# --- Self-contained: CRD post-hoc ---
set.seed(403)
p4_df <- tibble(
  treatment = factor(rep(c("A", "B", "C", "D"), each = 5)),
  y = 10 + c(0, 2, 4, 6)[rep(1:4, each = 5)] + rnorm(20, 0, 1.5)
)
p4_fit <- aov(y ~ treatment, data = p4_df)

p4_emm <- emmeans(p4_fit, ~ treatment)
pairs(p4_emm, adjust = "tukey")

## contrast estimate SE df t.ratio p.value
## A - B          -3.34 1.1 16  -3.044  0.0351
## A - C          -4.88 1.1 16  -4.453  0.0020
## A - D          -7.63 1.1 16  -6.953 <0.0001
## B - C          -1.55 1.1 16  -1.409  0.5120
## B - D          -4.29 1.1 16  -3.909  0.0062
## C - D          -2.74 1.1 16  -2.500  0.0980
##
## P value adjustment: tukey method for comparing a family of 4 estimates
```

6.5 4.5 CRD summary

Property	Value
Randomization	Full
Blocking	No
Error df	$N - k$
F-test denominator	MS_{Error}
Key assumption	Independence of units

7 Part 5 — RCBD: Randomized Complete Block Design

7.1 5.1 What it is

- One treatment factor.
- One blocking factor.
- Each treatment appears exactly once per block.
- Analysis: two-way ANOVA without interaction (usually).

Model: $y_{ij} = \mu + \tau_j + \beta_i + \varepsilon_{ij}$

- τ_j : treatment effect.
- β_i : block effect.
- Assumption: **no treatment $\times$ block interaction** (additive model).

7.2 5.2 Design and data

```
# --- Self-contained: RCBD data ---
set.seed(501)
```

```
p5_df <- expand.grid(
  block = factor(1:6),
  treatment = factor(c("A", "B", "C", "D"), levels = c("A", "B", "C", "D"))
) %>%
  mutate(y = 10 +
    c(A = 0, B = 2, C = 4, D = 6)[as.character(treatment)] +
    rep(c(-1, -0.5, 0, 0.5, 1, 1.5), each = 4) + # block effect
    rnorm(24, 0, 1))

head(p5_df)
```

```
##   block treatment      y
## 1     1          A 9.577281
## 2     2          A 9.617452
## 3     3          A 9.452007
## 4     4          A 9.225781
## 5     5          A 8.655195
## 6     6          A 8.066631
```

7.3 5.3 Fit and report

```
# --- Self-contained: RCBD ANOVA ---
set.seed(502)
p5_df <- expand.grid(
  block = factor(1:6),
  treatment = factor(c("A", "B", "C", "D"), levels = c("A", "B", "C", "D"))
) %>%
  mutate(y = 10 +
    c(A = 0, B = 2, C = 4, D = 6)[as.character(treatment)] +
    rep(c(-1, -0.5, 0, 0.5, 1, 1.5), each = 4) +
    rnorm(24, 0, 1))

p5_fit <- aov(y ~ treatment + block, data = p5_df)
summary(p5_fit)
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## treatment   3  223.83    74.61  85.585 1.16e-09 ***
## block       5   2.81     0.56   0.645    0.67
## Residuals   15  13.08     0.87
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Output:

```
           Df Sum Sq Mean Sq F value    Pr(>F)
treatment   3   ...
block       5   ...
Residuals   15   ...
```

Report: “RCBD ANOVA shows a significant treatment effect, $F(3, 15) = \dots$, $p = \dots$, after controlling for block effects.”

7.4 5.4 Testing for interaction

If you suspect treatment effects vary by block, fit the interaction model:

```
# --- Self-contained: RCBD with interaction test ---
set.seed(503)
p5_df <- expand.grid(
  block      = factor(1:6),
  treatment = factor(c("A", "B", "C", "D"))
) %>%
  mutate(y = 10 + c(A = 0, B = 2, C = 4, D = 6)[as.character(treatment)] +
    rep(c(-1, -0.5, 0, 0.5, 1, 1.5), each = 4) +
    rnorm(24, 0, 1))

p5_add <- aov(y ~ treatment + block, data = p5_df)
p5_inter <- aov(y ~ treatment * block, data = p5_df)
anova(p5_add, p5_inter)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ treatment + block
## Model 2: y ~ treatment * block
##   Res.Df    RSS Df Sum of Sq   F Pr(>F)
## 1      15 18.533
## 2       0  0.000 15    18.533 NaN    NaN
```

- $p > 0.05$: additive model adequate.
- $p < 0.05$: interaction present — report it or use Tukey’s test for non-additivity.

7.5 5.5 When to use RCBD

- You have a known nuisance factor that is likely to affect the response.
- You can create homogeneous blocks.
- The block effect is not of primary interest.

7.6 5.6 RCBD summary

Property	Value
Randomization	Within blocks
Blocking	Yes (1 factor)
Error df	$(b - 1)(k - 1)$
Model	Additive (no interaction assumed)
Benefit	Reduces error variance; increases power

8 Part 6 — Latin Square Design

8.1 6.1 What it is

- One treatment factor.
- Two blocking factors (rows and columns).
- Each treatment appears exactly once in each row and each column.
- Requires $k \times k$ layout (k treatments, k rows, k columns).
- Analysis: three-way ANOVA without interactions.

Model: $y_{ijk} = \mu + \alpha_i + \beta_j + \tau_k + \varepsilon_{ijk}$

8.2 6.2 Generate and analyze

```
# --- Self-contained: Latin Square (manual construction) ---
set.seed(601)
k <- 4

# Step 1: cyclic Latin Square base (each symbol appears once per row/col)
ls_base <- outer(1:k, 1:k, function(i, j) ((i + j - 2) %% k) + 1)

# Step 2: randomize rows, columns, and treatment labels
row_perm <- sample(1:k)
col_perm <- sample(1:k)
trt_map <- sample(LETTERS[1:k])

ls_rand <- ls_base[row_perm, col_perm]
ls_rand[] <- trt_map[ls_rand]

# Step 3: build long data frame from (row, col) grid
p6_df <- tibble(
  row = rep(1:k, each = k),
  col = rep(1:k, times = k)
) %>%
  mutate(
    trt = factor(ls_rand[cbind(row, col)]),
    row = factor(row),
    col = factor(col)
  )

# Step 4: simulate response (row effect + col effect + trt effect + noise)
p6_df <- p6_df %>%
  mutate(
    y = 10 +
      c(A = 0, B = 2, C = 4, D = 6)[as.character(trt)] +
      as.numeric(row) * 0.5 + as.numeric(col) * 0.3 +
      rnorm(k^2, 0, 1)
  )

kable(p6_df)
```

row	col	trt	y
1	1	B	13.26579
1	2	A	11.15585
1	3	D	15.80060
1	4	C	14.85297
2	1	D	16.53223
2	2	C	16.40389
2	3	B	13.77777
2	4	A	12.95502
3	1	C	13.96842
3	2	B	13.79019
3	3	A	12.44443
3	4	D	18.90118
4	1	A	13.10789

row	col	trt	y
4	2	D	17.40186
4	3	C	16.96531
4	4	B	14.93760

```
#Fit:

# --- Self-contained: Latin Square analysis ---
set.seed(602)
k <- 4
ls_base <- outer(1:k, 1:k, function(i, j) ((i + j - 2) %% k) + 1)
row_perm <- sample(1:k); col_perm <- sample(1:k); trt_map <- sample(LETTERS[1:k])
ls_rand <- ls_base[row_perm, col_perm]
ls_rand[] <- trt_map[ls_rand]

p6_df <- tibble(
  row = rep(1:k, each = k),
  col = rep(1:k, times = k)
) %>%
  mutate(
    trt = factor(ls_rand[cbind(row, col)]),
    row = factor(row),
    col = factor(col)
  ) %>%
  mutate(
    y = 10 +
      c(A = 0, B = 2, C = 4, D = 6)[as.character(trt)] +
      as.numeric(row) * 0.5 + as.numeric(col) * 0.3 +
      rnorm(k^2, 0, 1)
  )

p6_fit <- aov(y ~ row + col + trt, data = p6_df)
summary(p6_fit)

##              Df Sum Sq Mean Sq F value    Pr(>F)
## row              3    5.89   1.965    7.272  0.0201 *
## col              3    9.44   3.147   11.647  0.0065 **
## trt              3   85.29  28.429  105.233 1.41e-05 ***
## Residuals       6    1.62   0.270
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

9 Part 7 — BIBD: Balanced Incomplete Block Design

9.1 7.1 What it is

- Like RCBD, but each block contains only a **subset** of treatments.
- Each treatment appears in r blocks; each pair of treatments appears together in λ blocks.
- Used when you cannot fit all treatments in one block.

Parameters: - t = number of treatments. - k = block size ($k < t$). - r = number of replications per treatment. - b = number of blocks. - λ = number of blocks in which each pair appears. - Balance: $b \cdot k = t \cdot r$,

and $\lambda(t-1) = r(k-1)$.

9.2 7.2 Example

```
# --- Self-contained: BIBD design (manual construction) ---
# 7 treatments, block size 3, r = 3, b = 7, lambda = 1
set.seed(701)
p7_blocks <- list(
  c(1, 2, 3),
  c(1, 4, 5),
  c(2, 4, 6),
  c(3, 5, 6),
  c(1, 6, 7),
  c(2, 5, 7),
  c(3, 4, 7)
)

p7_df <- tibble(
  block = factor(rep(1:7, each = 3)),
  trt    = factor(unlist(p7_blocks))
) %>%
  mutate(
    y = 10 + as.numeric(trt) * 0.5 +
      rep(rnorm(7, 0, 1), each = 3) + rnorm(21, 0, 1)
  )

head(p7_df)
```

```
## # A tibble: 6 x 3
##   block trt      y
##   <fct> <fct> <dbl>
## 1 1     1     10.3
## 2 1     2     10.4
## 3 1     3     11.3
## 4 2     1      9.83
## 5 2     4     11.7
## 6 2     5     11.8
```

9.3 7.3 Analysis

BIBD requires a specific ANOVA that accounts for the incomplete block structure:

```
# --- Self-contained: BIBD analysis ---
set.seed(702)
p7_blocks <- list(
  c(1, 2, 3), c(1, 4, 5), c(2, 4, 6), c(3, 5, 6),
  c(1, 6, 7), c(2, 5, 7), c(3, 4, 7)
)

p7_df <- tibble(
  block = factor(rep(1:7, each = 3)),
  trt    = factor(unlist(p7_blocks))
) %>%
  mutate(y = 10 + as.numeric(trt) * 0.5 +
    rep(rnorm(7, 0, 1), each = 3) + rnorm(21, 0, 1))
```

```
p7_fit <- aov(y ~ trt + block, data = p7_df)
summary(p7_fit)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## trt          6 22.196   3.699    7.505 0.00601 **
## block        6  9.225   1.537    3.119 0.07005 .
## Residuals    8  3.943   0.493
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Note: The block SS is computed after adjusting for treatments (or vice versa). For proper BIBD analysis, use `agricolae::BIB.test()`:

```
agricolae::BIB.test(
  block = p7_df$block,
  trt    = p7_df$trt,
  y      = p7_df$y,
  test   = "tukey"
)
```

9.4 7.4 BIBD summary

Property	Value
Block size	Less than number of treatments
Requirement	Balanced (all pairs appear equally often)
Analysis	Adjusted for block structure
Use case	When block capacity is limited

10 Part 8 — Full Factorial Designs

10.1 8.1 What it is

- Two or more treatment factors crossed.
- All combinations of levels are tested.
- Each factor can be between-subjects or within-subjects.
- Analysis: factorial ANOVA.

10.2 8.2 2^k factorial

```
# --- Self-contained: 2^3 factorial ---
set.seed(801)
p8_df <- expand.grid(
  A = factor(c("-", "+")),
  B = factor(c("-", "+")),
  C = factor(c("-", "+"))
) %>%
  slice(rep(1:n(), each = 5)) %>%
  mutate(
    y = 10 + 2 * (A == "+") + 1 * (B == "+") - 1.5 * (C == "+") +
      0.8 * (A == "+") * (B == "+") + rnorm(n(), 0, 1)
```

```
)

p8_fit <- aov(y ~ A * B * C, data = p8_df)
summary(p8_fit)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## A              1  91.69   91.69   97.489 3.09e-11 ***
## B              1  13.16   13.16   13.989 0.000722 ***
## C              1  19.82   19.82   21.079 6.49e-05 ***
## A:B            1   5.19    5.19    5.518 0.025165 *
## A:C            1   1.53    1.53    1.625 0.211524
## B:C            1   0.17    0.17    0.183 0.671441
## A:B:C          1   0.27    0.27    0.286 0.596650
## Residuals     32  30.10    0.94
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Output: One row for each main effect (A, B, C), each 2-way interaction (A:B, A:C, B:C), and the 3-way interaction (A:B:C).

Report: “Factorial ANOVA shows a significant main effect of A, $F(1, 32) = \dots$, $p < \dots$, and a significant A×B interaction, $F(1, 32) = \dots$, $p < \dots$. No other effects are significant.”

10.3 8.3 2³ with blocking

```
# --- Self-contained: 2^3 factorial with blocks ---
set.seed(802)
p8_df <- expand.grid(
  block = factor(1:4),
  A = factor(c("-", "+")),
  B = factor(c("-", "+")),
  C = factor(c("-", "+"))
) %>%
  mutate(y = 10 + 2 * (A == "+") + (B == "+") +
    rnorm(n(), 0, 1))

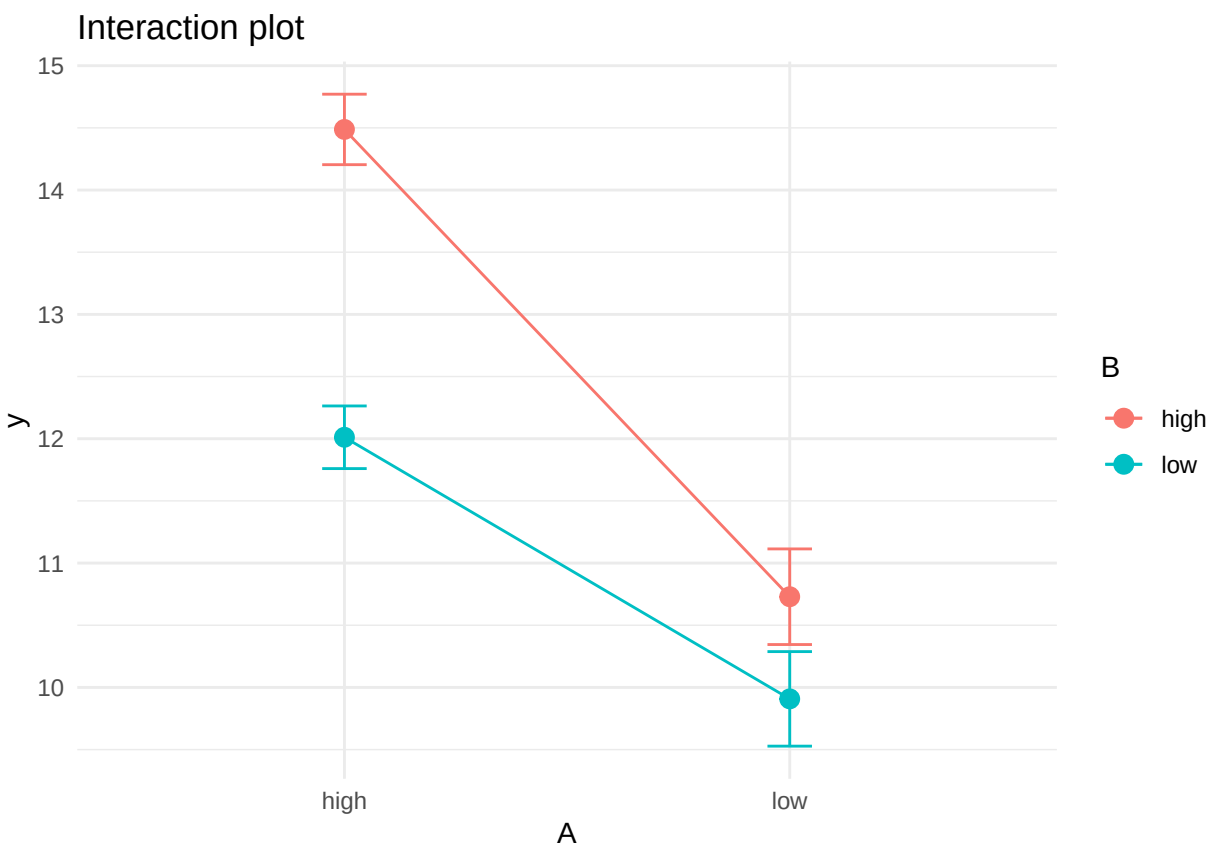
p8_fit <- aov(y ~ block + A * B * C, data = p8_df)
summary(p8_fit)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## block         3   0.82    0.27   0.655   0.589
## A              1  33.77   33.77  81.426 1.13e-08 ***
## B              1  19.28   19.28  46.486 9.66e-07 ***
## C              1   0.96    0.96   2.308   0.144
## A:B            1   0.26    0.26   0.617   0.441
## A:C            1   0.24    0.24   0.581   0.455
## B:C            1   0.73    0.73   1.752   0.200
## A:B:C          1   0.00    0.00   0.001   0.978
## Residuals     21   8.71    0.41
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

10.4 8.4 Interaction plots

```
# --- Self-contained: interaction plot ---
set.seed(803)
p8_df <- expand_grid(
  A = factor(c("low", "high")),
  B = factor(c("low", "high"))
) %>%
  slice(rep(1:n(), each = 10)) %>%
  mutate(y = 10 + 2 * (A == "high") + (B == "high") +
    1.5 * (A == "high") * (B == "high") + rnorm(n(), 0, 1))

ggplot(p8_df, aes(A, y, color = B, group = B)) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun = mean, geom = "line") +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.1) +
  theme_minimal() +
  labs(title = "Interaction plot")
```



Look for: crossing or diverging lines → interaction. Parallel lines → no interaction.

10.5 8.5 Factorial design summary

Property	Value
Factors	2+

Property	Value
Levels per factor	Any (2, 3, ...)
Design type	Full factorial
Total treatment combinations	Product of levels
Analysis	Factorial ANOVA
Key feature	Tests interactions

10.6 8.6 When to use

- You want to test main effects AND interactions.
- You can afford to run all combinations.
- You have enough replicates per cell.

11 Part 9 — Fractional Factorial Designs

11.1 9.1 What it is

- A **subset** of the full factorial combinations.
- Used when the number of factors is large and full factorial is too expensive.
- Some effects are **aliased** (confounded) with others.
- Denoted 2^{k-p} — a $1/2^p$ fraction of the 2^k design.

11.2 9.2 Resolution

The **resolution** of a fractional factorial describes which effects are aliased:

Resolution	Aliasing rule
III	Main effects aliased with 2-way interactions
IV	Main effects aliased with 3-way interactions; 2-way aliased with 2-way
V	Main effects aliased with 4-way interactions; 2-way aliased with 3-way

Rule of thumb: Use Resolution V if possible, Resolution IV for screening, Resolution III for very early screening.

11.3 9.3 Generate the design

```
# --- Self-contained: 2^(4-1) fractional factorial ---
set.seed(901)
p9_design <- FrF2::FrF2(nruns = 8, nfactors = 4, replications = 3)
p9_df <- as_tibble(p9_design) %>%
  mutate(
    y = 10 + 2 * (A == "1") + (B == "1") - (C == "1") +
      rnorm(n(), 0, 1)
  )
head(p9_df)
```

```
## # A tibble: 6 x 5
##   A     B     C     D     y
```

```
##   <fct> <fct> <fct> <fct> <dbl>
## 1 1      -1     -1     1      13.7
## 2 1       1     -1    -1      12.8
## 3 -1      1     -1     1      10.9
## 4 -1     -1    -1    -1       8.65
## 5 1      -1     1     -1      12.2
## 6 -1      1     1     -1      11.6
```

What the columns are:

A, B, C, D are the four factors (levels -1 / 1).

run is the standard order within each replicate.

replications = 3 gives 3 independent runs per treatment combination.

The design is Resolution IV because the generator is $D = A*B*C$, giving defining relation $I = ABCD$.

9.4 Aliasing structure

The aliasing is fixed by the generator. For a $2^{(4-1)}$ design with $D = ABC$: Effect Aliased with A $B \times C \times D$
 B $A \times C \times D$ C $A \times B \times D$ D $A \times B \times C$ A $B \times C \times D$ A $C \times B \times D$ A $D \times B \times C$

Why: The defining relation is $I = ABCD$. Multiplying any effect by the defining relation gives its alias.

To confirm the resolution from R:

```
# --- Self-contained: confirm resolution from the design ---
set.seed(901)
p9_design <- FrF2::FrF2(nruns = 8, nfactors = 4)

# Fit a model that includes all main effects and 2-way interactions
p9_wide <- as_tibble(p9_design) %>%
  mutate(y = 10 + 2 * (A == "1") + (B == "1") - (C == "1") + rnorm(n(), 0, 1))

p9_saturated <- lm(y ~ (A + B + C + D)^2, data = p9_wide)

# stats::alias shows which effects are aliased
stats::alias(p9_saturated)
```

```
## Model :
## y ~ (A + B + C + D)^2
##
## Complete :
##      (Intercept) A1 B1 C1 D1 A1:B1 A1:C1 A1:D1
## B1:C1 0          0 0 0 0 0      0      1
## B1:D1 0          0 0 0 0 0      1      0
## C1:D1 0          0 0 0 0 1      0      0
```

What the output tells you: The Complete column lists which effects are aliased with each other. For a Res IV design, main effects are aliased only with 3-way interactions (which are assumed negligible), so main-effect p-values are trustworthy.

9.5 Analysis

```
# --- Self-contained: fractional factorial analysis ---
set.seed(902)
p9_design <- FrF2::FrF2(nruns = 8, nfactors = 4, replications = 3)
p9_df <- as_tibble(p9_design) %>%
```

```
mutate(
  y = 10 + 2 * (A == "1") + (B == "1") - (C == "1") + rnorm(n(), 0, 1)
)

# Fit main effects only (higher-order interactions assumed negligible)
p9_fit <- lm(y ~ A + B + C + D, data = p9_df)
anova(p9_fit)
```

```
## Analysis of Variance Table
```

```
##
```

```
## Response: y
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## A             1 34.113   34.113  39.9617 4.568e-06 ***
## B             1  4.851    4.851   5.6832 0.027714 *
## C             1  8.993    8.993  10.5350 0.004254 **
## D             1  0.065    0.065   0.0763 0.785324
## Residuals    19 16.219    0.854
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation: “Main effect A is significant. Since the design is Resolution IV, A is aliased only with the B×C×D interaction, which is assumed negligible. Main-effect tests are therefore valid.”

11.4 9.6 Fractional factorial summary

Property	Value
Total runs	2^{k-p}
Resolution	III, IV, or V
Key trade-off	Efficiency vs aliasing
Analysis	Regression on main effects and unaliased interactions
Use case	Screening many factors

11.5 9.7 Blocking in factorial designs

Factorial designs can be blocked like any other design:

```
# --- Self-contained: blocked 2^2 design ---
set.seed(903)
p9_df <- expand.grid(
  block = factor(1:3),
  A = factor(c("-", "+")),
  B = factor(c("-", "+"))
) %>%
  slice(rep(1:n(), each = 2)) %>%
  mutate(y = 10 + (A == "+") + (B == "+") + rnorm(n(), 0, 1))

p9_fit <- aov(y ~ block + A * B, data = p9_df)
summary(p9_fit)
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## block       2  3.324    1.662    1.697  0.211
## A           1  5.226    5.226    5.335  0.033 *
## B           1  2.916    2.916    2.977  0.102
```

```
## A:B          1  0.193   0.193   0.197  0.663
## Residuals    18 17.630   0.979
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

In fractional factorials, blocks are usually chosen so that block effects are aliased with higher-order interactions (not main effects).

12 Part 10 — Split-Plot Designs

12.1 10.1 What it is

- Used when some factors are **hard to change** (whole-plot factors) and others are **easy to change** (split-plot factors).
- The whole-plot factor is applied to large units (whole plots).
- The split-plot factor is applied to smaller units within each whole plot.
- Each whole-plot factor level is applied to a whole-plot; each split-plot factor is applied within it.

Classic example: Agriculture — irrigation method (hard to change) is the whole-plot factor; fertilizer type (easy to change) is the split-plot factor.

12.2 10.2 Two error terms

Split-plot designs have **two error strata**:

- **Whole-plot error** — variation among whole plots within a whole-plot treatment.
- **Split-plot error** — variation among subplots within whole plots.

Correct **F**-tests:

Effect	Denominator
Whole-plot main effect	Whole-plot error
Split-plot main effect	Split-plot error
Whole × split interaction	Split-plot error

12.3 10.3 Generate and analyze

```
# --- Self-contained: split-plot design ---
set.seed(1001)
p10_df <- expand.grid(
  block      = factor(1:4),
  whole      = factor(c("W1", "W2")),
  split      = factor(c("S1", "S2", "S3"))
) %>%
  mutate(y = 10 + 2 * (whole == "W2") + (split == "S2") +
    2 * (split == "S3") +
    1.5 * (whole == "W2") * (split == "S3") +
    rep(rnorm(8, 0, 1.5), each = 3) +
    rnorm(24, 0, 1))

p10_fit <- aov(y ~ whole * split + Error(block / whole), data = p10_df)
summary(p10_fit)
```



```
##
## Error: block
##           Df Sum Sq Mean Sq F value Pr(>F)
## Residuals  3  9.483   3.161
##
## Error: block:whole
##           Df Sum Sq Mean Sq F value Pr(>F)
## whole      1  34.58   34.58  17.33 0.0252 *
## Residuals  3   5.99    2.00
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Error: Within
##           Df Sum Sq Mean Sq F value Pr(>F)
## split      2  36.98  18.490   7.235 0.00868 **
## whole:split 2  24.95  12.475   4.881 0.02811 *
## Residuals 12  30.67   2.556
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Output has two strata:

```
Error: block:whole
           Df Sum Sq Mean Sq F value Pr(>F)
whole      1  ...      ...
Residuals  6  ...
```

```
Error: Within
           Df Sum Sq Mean Sq F value Pr(>F)
split      2  ...      ...
whole:split 2  ...      ...
Residuals 18  ...      ...
```

- Whole-plot main effect tested against Error: block:whole residual.
- Split-plot main effect and interaction tested against Error: Within residual.

12.4 10.4 Split-plot with lmer

```
# --- Self-contained: split-plot with lmer ---
set.seed(1002)
p10_df <- expand.grid(
  block = factor(1:4),
  whole = factor(c("W1", "W2")),
  split = factor(c("S1", "S2", "S3"))
) %>%
  mutate(y = 10 + 2 * (whole == "W2") + (split == "S2") +
    2 * (split == "S3") +
    1.5 * (whole == "W2") * (split == "S3") +
    rep(rnorm(8, 0, 1.5), each = 3) +
    rnorm(24, 0, 1))

p10_fit_lmer <- lmer(y ~ whole * split + (1 | block:whole), data = p10_df)
anova(p10_fit_lmer)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
```

```
##           Sum Sq Mean Sq NumDF DenDF F value Pr(>F)
## whole      2.7015  2.7015     1    18  1.1651 0.29466
## split     12.2443  6.1221     2    18  2.6404 0.09874 .
## whole:split  7.7159  3.8580     2    18  1.6639 0.21724
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

12.5 10.5 Split-plot summary

Property	Value
Whole-plot factor	Hard to change
Split-plot factor	Easy to change
Randomization	Whole plots then split plots
Error terms	2 (whole-plot + split-plot)
F-test denominators	Different for each effect
Key risk	Using the wrong error term

12.6 10.6 Split-split-plot and strip-plot

- **Split-split-plot:** a split-plot factor is further split into sub-sub-plots. Three error terms.
- **Strip-plot:** whole-plot treatments applied in strips crossing each other. Two whole-plot error terms.

13 Part 11 — Nested vs Crossed Designs

13.1 11.1 Crossed design

- Every level of factor A is tested with every level of factor B.
- Analysis: `aov(y ~ A * B)`.

Example: 3 drugs $\times$ 2 doses — every drug is tested at every dose.

13.2 11.2 Nested design

- Levels of B are **unique** within each level of A.
- B is nested within A. Denoted B(A).
- Analysis: `aov(y ~ A + Error(A/B))` or `lmer(y ~ A + (1|A:B))`.

Example: Schools and classes — each class belongs to exactly one school.

13.3 11.3 Example

```
# --- Self-contained: nested design ---
set.seed(1101)
p11_df <- tibble(
  school = factor(rep(c("S1", "S2", "S3"), each = 6)),
  teacher = factor(rep(1:6, times = 3)), # unique to each school
  y = 10 + as.numeric(school) * 2 +
    rep(rnorm(3, 0, 1), each = 6) +
    rnorm(18, 0, 1)
)
```

```
# Incorrect: crossed
p11_fit_crossed <- aov(y ~ school + teacher, data = p11_df)
# Correct: nested
p11_fit_nested <- aov(y ~ school + Error(school / teacher), data = p11_df)
summary(p11_fit_nested)
```

```
##
## Error: school
##      Df Sum Sq Mean Sq
## school  2  68.45   34.22
##
## Error: school:teacher
##      Df Sum Sq Mean Sq F value Pr(>F)
## Residuals 15  18.32    1.221
```

13.4 11.4 Nested vs crossed summary

	Crossed	Nested
Design	$A \times B$	$A + B(A)$
Every $A \times B$ seen	Yes	No (B unique to A)
R formula	$A * B$	$A + \text{Error}(A/B)$
Use case	Factorial	Hierarchical

14 Part 12 — Crossover Designs

14.1 12.1 What it is

- Each subject receives **multiple treatments** in sequence.
- Common in clinical trials: each patient gets treatment A, then B (or B, then A).
- Uses subjects as their own controls.

Model (2×2 crossover): $y_{ijk} = \mu + \text{treatment}_k + \text{period}_j + \text{subject}_i + \varepsilon_{ijk}$

14.2 12.2 2×2 crossover

```
# --- Self-contained: 2x2 crossover ---
set.seed(1201)
p12_df <- expand_grid(
  subject = factor(1:20),
  period = factor(c("P1", "P2"))
) %>%
  mutate(
    sequence = rep(rep(c("AB", "BA"), each = 2), 10),
    treatment = case_when(
      sequence == "AB" & period == "P1" ~ "A",
      sequence == "AB" & period == "P2" ~ "B",
      sequence == "BA" & period == "P1" ~ "B",
      sequence == "BA" & period == "P2" ~ "A"
    ),
    y = 10 + 2 * (treatment == "B") + rep(rnorm(20, 0, 1), each = 2) +
```

```

    rnorm(40, 0, 1)
  )

p12_fit <- lmer(y ~ treatment + period + (1 | subject), data = p12_df)
anova(p12_fit)

## Type III Analysis of Variance Table with Satterthwaite's method
##           Sum Sq Mean Sq NumDF DenDF F value    Pr(>F)
## treatment  41.688  41.688     1    37 25.4852 1.215e-05 ***
## period      0.798   0.798     1    37  0.4876  0.4894
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

14.3 12.3 Crossover summary

Property	Value
Randomization	Sequence order
Blocking	Subject
Key risk	Carryover effects (need washout)
Analysis	Mixed model with subject random effect

15 Part 13 — Repeated Measures ANOVA

15.1 13.1 What it is

- Same subjects measured multiple times.
- Time is a **within-subject** factor.
- Analysis: `aov(y ~ time + Error(subject/time))` or `lmer`.

15.2 13.2 Fit and report

```

# --- Self-contained: one-factor RM ANOVA ---
set.seed(1301)
p13_df <- expand.grid(
  subject = factor(1:10),
  time    = factor(c("t1", "t2", "t3"), levels = c("t1", "t2", "t3"))
) %>%
  mutate(y = 10 + as.numeric(time) * 2 +
    rep(rnorm(10, 0, 1), each = 3) +
    rnorm(30, 0, 1))

p13_fit <- aov(y ~ time + Error(subject / time), data = p13_df)
summary(p13_fit)

##
## Error: subject
##           Df Sum Sq Mean Sq F value Pr(>F)
## Residuals   9  12.35   1.373
##
## Error: subject:time

```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## time         2  49.57   24.787    12.3 0.00043 ***
## Residuals    18   36.28    2.016
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

15.3 13.3 Reading the output

```
Error: subject
      Df Sum Sq Mean Sq F value Pr(>F)
Residuals 9    ...
```

```
Error: subject:time
      Df Sum Sq Mean Sq F value Pr(>F)
time     2    ...      ...
Residuals 18    ...
```

Report the F-test in the Error: `subject:time` stratum. The top stratum is between-subject variability.

15.4 13.4 Sphericity and corrections

Sphericity: equal variances of all pairwise differences. Violated → inflated Type I error.

```
# --- Self-contained: afex sphericity corrections ---
set.seed(1302)
p13_df <- expand.grid(
  subject = factor(1:10),
  time    = factor(c("t1", "t2", "t3"), levels = c("t1", "t2", "t3"))
) %>%
  mutate(y = 10 + as.numeric(time) * 2 +
    rep(rnorm(10, 0, 1), each = 3) +
    rnorm(30, 0, 1))

p13_afex <- afex::aov_ez(
  id      = "subject",
  dv      = "y",
  data    = as.data.frame(p13_df),
  within  = "time"
)
summary(p13_afex)
```

```
##
## Univariate Type III Repeated-Measures ANOVA Assuming Sphericity
##
##           Sum Sq num Df Error SS den Df  F value    Pr(>F)
## (Intercept) 5908.7      1  14.643      9 3631.677 4.808e-13 ***
## time         61.7      2   54.875     18  10.127  0.001131 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Mauchly Tests for Sphericity
##
##      Test statistic  p-value
## time      0.42304 0.032026
##
```

```
##
## Greenhouse-Geisser and Huynh-Feldt Corrections
## for Departure from Sphericity
##
##      GG eps Pr(>F[GG])
## time 0.63413 0.005988 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      HF eps  Pr(>F[HF])
## time 0.6908272 0.004613612
```

afex reports **Greenhouse-Geisser** and **Huynh-Feldt** corrections automatically.

15.5 13.5 lmer alternative

```
# --- Self-contained: lmer for RM ---
set.seed(1303)
p13_df <- expand.grid(
  subject = factor(1:10),
  time    = factor(c("t1", "t2", "t3"), levels = c("t1", "t2", "t3"))
) %>%
  mutate(y = 10 + as.numeric(time) * 2 +
         rep(rnorm(10, 0, 1), each = 3) + rnorm(30, 0, 1))

p13_fit_lmer <- lmer(y ~ time + (1 | subject), data = p13_df)
anova(p13_fit_lmer)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##      Sum Sq Mean Sq NumDF DenDF F value    Pr(>F)
## time 60.348  30.174      2    27  19.532 5.675e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

When to prefer lmer: missing data, unbalanced, random slopes.

16 Part 14 — Mixed Designs (Between + Within)

16.1 14.1 What it is

- One or more **between-subject** factors (subjects in one group only).
- One or more **within-subject** factors (each subject measured multiple times).
- Analysis: `aov(y ~ between * within + Error(subject/within))`.

16.2 14.2 Fit and report

```
# --- Self-contained: mixed design ---
set.seed(1401)
p14_df <- expand.grid(
  subject = factor(1:20),
  time    = factor(c("pre", "post"), levels = c("pre", "post"))
) %>%
  mutate(
```

```

group = factor(rep(rep(c("control", "treat"), each = 10), 2)),
y = 10 + 2 * (time == "post") + 3 * (group == "treat") +
  2 * (time == "post") * (group == "treat") +
  rep(rnorm(20, 0, 1), each = 2) + rnorm(40, 0, 1)
)

p14_fit <- aov(y ~ group * time + Error(subject / time), data = p14_df)
summary(p14_fit)

```

```

##
## Error: subject
##           Df Sum Sq Mean Sq F value    Pr(>F)
## group      1 124.61  124.61    50.38 1.29e-06 ***
## Residuals 18  44.52    2.47
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Error: subject:time
##           Df Sum Sq Mean Sq F value    Pr(>F)
## time        1  64.69   64.69   29.43 3.74e-05 ***
## group:time   1   4.37    4.37    1.99   0.175
## Residuals 18  39.56    2.20
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

16.3 14.3 Reading the output

```

Error: subject
      Df Sum Sq Mean Sq F value Pr(>F)
group   1   ...      ...
Residuals 18   ...

```

```

Error: subject:time
      Df Sum Sq Mean Sq F value Pr(>F)
time    1   ...      ...
group:time 1   ...      ...
Residuals 18   ...

```

Effect	Stratum
group (between)	Error: subject
time (within)	Error: subject:time
group:time	Error: subject:time

16.4 14.4 Mixed-design summary

Property	Value
Between-subject factor	e.g., group
Within-subject factor	e.g., time
Randomization	Subjects to groups
Multiple error terms	Yes
F-test denominators	Depends on effect

17 Part 15 — Fixed vs Random Effects

17.1 15.1 The distinction

Aspect	Fixed effect	Random effect
Levels	All levels of interest	Random sample from population
Inference	To the tested levels only	To the population
Effect on variance	Affects mean	Affects variance
Typical examples	Treatment, dose, sex	Subject, batch, site
R syntax (<code>lmer</code>)	<code>y ~ treatment</code>	<code>y ~ (1 subject)</code>

17.2 15.2 Why it matters

- **F-test denominator changes.** Fixed effects are often tested against residual error; random effects may have different error terms.
- **Estimation method differs.** Fixed effects use OLS; random effects use GLS / REML.
- **Interpretation differs.** Fixed: “the effect of treatment A vs B.” Random: “the variance among subjects.”

17.3 15.3 Mixed models

Models with both fixed and random effects are called **mixed models**.

```
# --- Self-contained: mixed model ---
set.seed(1501)
p15_df <- expand.grid(
  subject = factor(1:20),
  time    = factor(c("t1", "t2", "t3"), levels = c("t1", "t2", "t3"))
) %>%
  mutate(y = 10 + as.numeric(time) * 1.5 +
    rep(rnorm(20, 0, 1), each = 3) + rnorm(60, 0, 1))

# time is fixed; subject is random
p15_fit <- lmer(y ~ time + (1 | subject), data = p15_df)
summary(p15_fit)

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: y ~ time + (1 | subject)
## Data: p15_df
##
## REML criterion at convergence: 206.5
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.30283 -0.68361  0.00598  0.74440  2.01845
##
## Random effects:
## Groups   Name                Variance Std.Dev.
## subject (Intercept)  0.000      0.000
## Residual                    1.872      1.368
```



```
## Number of obs: 60, groups:  subject, 20
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)  11.3861      0.3059  57.0000   37.219 < 2e-16 ***
## timet2       1.8759      0.4326  57.0000    4.336 5.98e-05 ***
## timet3       3.4153      0.4326  57.0000    7.894 1.03e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) timet2
## timet2 -0.707
## timet3 -0.707  0.500
## optimizer (nloptwrap) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

17.4 15.4 Rule of thumb

- If you care about **specific levels** → fixed.
- If the levels are a **sample** from a larger population → random.
- If you want to **generalize** beyond the tested levels → random.
- If you want to test **specific contrasts** → fixed.

17.5 15.5 Mixed-model terminology

Term	Meaning
Fixed effect	Population-level effect
Random intercept	Each group has its own baseline
Random slope	Each group has its own slope on a predictor
Crossed random effects	Groups that can appear in multiple categories
Nested random effects	Groups within groups

18 Part 16 — Expected Mean Squares (EMS)

18.1 16.1 Why EMS matters

The **Expected Mean Square** tells you which error term to use as the denominator for each F-test. Choosing the wrong denominator gives wrong p-values.

18.2 16.2 EMS table for common designs

18.2.1 One-way ANOVA (fixed treatment)

Source	df	EMS
Treatment	$k - 1$	$\sigma^2 + n \sum \tau_j^2 / (k - 1)$
Error	$N - k$	σ^2

F-test: $F = MS_{\text{Treat}} / MS_{\text{Error}}$.

18.2.2 One-way ANOVA (random treatment)

Source	df	EMS
Treatment	$k - 1$	$\sigma^2 + n\sigma_\tau^2$
Error	$N - k$	σ^2

F-test: $F = MS_{\text{Treat}}/MS_{\text{Error}}$.

Note: For one-way, fixed and random give the same F-test. The difference shows up in interpretation, not the ANOVA table.

18.2.3 Two-way factorial (A, B fixed)

Source	df	EMS
A	$a - 1$	$\sigma^2 + nb \sum \alpha_i^2 / (a - 1)$
B	$b - 1$	$\sigma^2 + na \sum \beta_j^2 / (b - 1)$
AB	$(a - 1)(b - 1)$	$\sigma^2 + n \sum (\alpha\beta)_{ij}^2 / (a - 1)(b - 1)$
Error	$ab(n - 1)$	σ^2

F-tests: All tested against MS_{Error} .

18.2.4 Two-way factorial (A fixed, B random)

Source	df	EMS
A	$a - 1$	$\sigma^2 + n\sigma_{\alpha\beta}^2 + nb \sum \alpha_i^2 / (a - 1)$
B	$b - 1$	$\sigma^2 + na\sigma_\beta^2$
AB	$(a - 1)(b - 1)$	$\sigma^2 + n\sigma_{\alpha\beta}^2$
Error	$ab(n - 1)$	σ^2

F-tests: - A: $F = MS_A/MS_{AB}$ (because MS_{AB} has the extra term) - B: $F = MS_B/MS_{\text{Error}}$ - AB: $F = MS_{AB}/MS_{\text{Error}}$

Key takeaway: For mixed models with a random factor, main effects are often tested against the interaction MS instead of the residual.

18.2.5 Split-plot (A whole-plot fixed, B split-plot fixed)

Source	df	EMS	F denominator
A	$a - 1$	$\sigma_W^2 + b\sigma_{\text{WP}}^2 + rb \sum \alpha_i^2 / (a - 1)$	Whole-plot error
WP error	$(r - 1)(a - 1)$	$\sigma_W^2 + b\sigma_{\text{WP}}^2$	—
B	$b - 1$	$\sigma_W^2 + ra \sum \beta_j^2 / (b - 1)$	Split-plot error
AB	$(a - 1)(b - 1)$	$\sigma_W^2 + r \sum (\alpha\beta)^2 / (a - 1)(b - 1)$	Split-plot error
SP error	$a(b - 1)(r - 1)$	σ_W^2	—

18.3 16.3 EMS summary

Design	Effect	Denominator
One-way fixed	Treatment	Error
Two-way all fixed	A, B, AB	Error
Two-way A fixed, B random	A	AB
Two-way A fixed, B random	B, AB	Error
Split-plot	Whole-plot factor	Whole-plot error
Split-plot	Split-plot factor, interaction	Split-plot error
Nested	Fixed	Depends on structure
RM ANOVA	Time	Subject $\times$ time error

Rule: If a random effect appears in the EMS of an effect **but not** in the EMS of another, the latter can be used as the error term.

19 Part 17 — Power and Sample Size

19.1 17.1 Key parameters

- **Effect size:** standardized difference (Cohen's d , f , s^2).
- **Significance level** : usually 0.05.
- **Power** $1 - \alpha$: usually 0.80 (80%).
- **Sample size n :** what you solve for.

19.2 17.2 Power for one-way ANOVA

```
# --- Self-contained: one-way power ---
pwr.anova.test(k = 3, n = 20, f = 0.25, sig.level = 0.05)
```

```
##
##      Balanced one-way analysis of variance power calculation
##
##              k = 3
##              n = 20
##              f = 0.25
##      sig.level = 0.05
##      power = 0.3744311
##
## NOTE: n is number in each group
```

Arguments: k = groups, n = per group, f = Cohen's f .

Interpretation: “With 3 groups, 20 per group, and medium effect size ($f = 0.25$), power is 55%.”

19.3 17.3 Sample size needed

```
# --- Self-contained: sample size for 80% power ---
pwr.anova.test(k = 3, f = 0.25, sig.level = 0.05, power = 0.80)
```

```
##
##      Balanced one-way analysis of variance power calculation
```

```
##
##          k = 3
##          n = 52.3966
##          f = 0.25
##      sig.level = 0.05
##          power = 0.8
##
## NOTE: n is number in each group
```

Output: $n = 52.4 \rightarrow$ need 53 per group.

19.4 17.4 Power for factorial designs

```
# --- Self-contained: power for 2x2 factorial ---
pwr.f2.test(u = 1, v = 76, f2 = 0.0625, sig.level = 0.05)
```

```
##
##      Multiple regression power calculation
##
##          u = 1
##          v = 76
##          f2 = 0.0625
##      sig.level = 0.05
##          power = 0.5870888
```

Arguments: - u: numerator df (1 for a 2-level main effect). - v: denominator df (residual df). - f2 = f^2 where f is Cohen's f.

19.5 17.5 Power for RCBD

Blocking usually increases power by reducing error variance. Rule of thumb: if blocks explain 20% of variance, blocking is worth it.

19.6 17.6 Power for RM

```
# --- Self-contained: power for paired design ---
pwr.t.test(d = 0.5, sig.level = 0.05, power = 0.80, type = "paired")
```

```
##
##      Paired t test power calculation
##
##          n = 33.36713
##          d = 0.5
##      sig.level = 0.05
##          power = 0.8
##      alternative = two.sided
##
## NOTE: n is number of *pairs*
```

19.7 17.7 Post-hoc power

```
# --- Self-contained: post-hoc power ---
pwr.anova.test(k = 3, n = 20, f = 0.30, sig.level = 0.05)
```

```
##
```

```
##      Balanced one-way analysis of variance power calculation
##
##          k = 3
##          n = 20
##          f = 0.3
##      sig.level = 0.05
##          power = 0.5139776
##
## NOTE: n is number in each group
```

Warning: Post-hoc power is monotonically related to the p-value — it adds little information. Prefer reporting effect size and CI.

19.8 17.8 Effect size benchmarks

Measure	Small	Medium	Large
Cohen's d	0.20	0.50	0.80
Cohen's f	0.10	0.25	0.40
²	0.01	0.06	0.14
²	0.01	0.06	0.14

20 Part 18 — Blocking Efficiency

20.1 18.1 Why blocking helps

Blocking removes the block-to-block variation from the error term. If the block effect is large, the residual SS shrinks, so the F-test for treatment is more powerful.

20.2 18.2 Efficiency formula

$$\text{Relative Efficiency} = \frac{MS_{\text{Error(CRD)}}}{MS_{\text{Error(RCBD)}}} = \frac{(b-1)MS_{\text{Block}} + b(k-1)MS_{\text{Error}}}{(bk-1)MS_{\text{Error}}}$$

where b = blocks, k = treatments.

Interpretation: If RE = 2, RCBD is equivalent to running a CRD with **twice** the sample size.

20.3 18.3 Computing in R

```
# --- Self-contained: blocking efficiency ---
set.seed(1801)
p18_df <- expand.grid(
  block      = factor(1:6),
  treatment = factor(c("A", "B", "C", "D"))
) %>%
  mutate(y = 10 + c(A = 0, B = 2, C = 4, D = 6)[as.character(treatment)] +
    rep(c(-1, -0.5, 0, 0.5, 1, 1.5), each = 4) +
    rnorm(24, 0, 1))

p18_rcbd <- aov(y ~ treatment + block, data = p18_df)
p18_crd  <- aov(y ~ treatment, data = p18_df)
```

```
anova(p18_crd)

## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## treatment  3 250.986   83.662  82.738 1.899e-11 ***
## Residuals 20  20.223    1.011
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(p18_rcbd)

## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## treatment  3 250.986   83.662  85.5535 1.163e-09 ***
## block       5   5.555    1.111   1.1361   0.3839
## Residuals 15  14.668    0.978
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
b <- 6; k <- 4
MSB <- anova(p18_rcbd)["block", "Mean Sq"]
MSE <- anova(p18_rcbd)["Residuals", "Mean Sq"]
RE <- ((b - 1) * MSB + b * (k - 1) * MSE) / ((b * k - 1) * MSE)
RE
```

```
## [1] 1.02959
```

Report: “The blocking efficiency is 2.5, meaning the RCBD is equivalent to a CRD with $2.5\times$ the sample size.”

20.4 18.4 When to block

- Block if the nuisance factor explains $> 20\%$ of the total variation.
- Do not block if blocks are small or inconsistent.
- You can always test the block effect and decide.

21 Part 19 — Diagnostics

21.1 19.1 Assumptions

1. **Independence** of residuals.
2. **Normality** of residuals.
3. **Equal variances** across groups.
4. **Additivity** (for RCBD, split-plot).

21.2 19.2 Checking in R

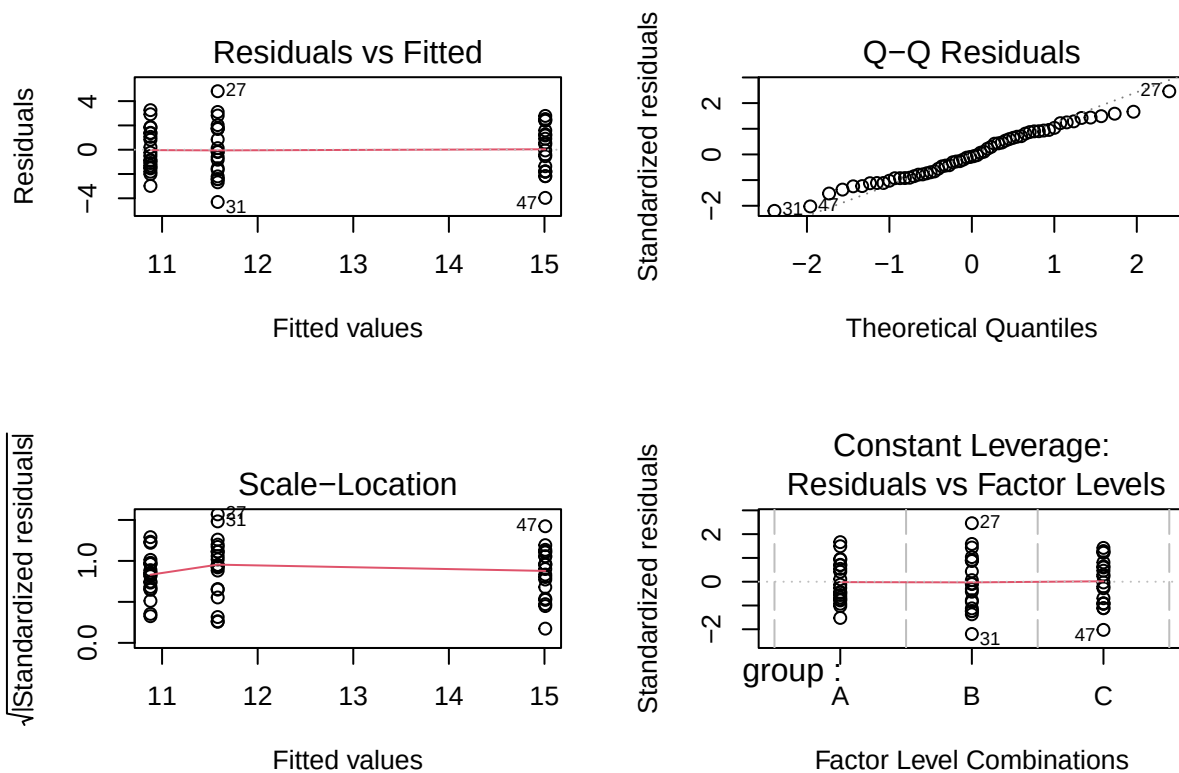
```
# --- Self-contained: diagnostic checks ---
set.seed(1901)
```

```

p19_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y      = c(rnorm(20, 10, 2), rnorm(20, 12, 2), rnorm(20, 15, 2))
)
p19_fit <- aov(y ~ group, data = p19_df)

# Plots
par(mfrow = c(2, 2))
plot(p19_fit)

```



```

par(mfrow = c(1, 1))

# Shapiro-Wilk (normality)
shapiro.test(residuals(p19_fit))

##
##  Shapiro-Wilk normality test
##
## data:  residuals(p19_fit)
## W = 0.98315, p-value = 0.5749

# Levene (equal variances)
car::leveneTest(y ~ group, data = p19_df)

## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value Pr(>F)
## group 2   0.804 0.4525

```

21.3 19.3 Interpreting the plots

Plot	What to look for
Residuals vs Fitted	No pattern; no funnel shape
Normal Q-Q	Points on the diagonal
Scale-Location	Flat line
Residuals vs Leverage	No influential points

21.4 19.4 What to do when assumptions fail

Problem	Fix
Non-normality	Transform (log, sqrt, Box-Cox); non-parametric test
Unequal variances	Welch ANOVA; Games-Howell post-hoc
Non-additivity in RCBD	Tukey's test for non-additivity; interaction model
Outliers	Check for data errors; report with/without
Non-independence	Mixed model with random effect

21.5 19.5 Diagnostic report template

“Shapiro-Wilk on residuals gives $p = 0.31$, Levene's test gives $p = 0.42$. The Q-Q plot shows points on the diagonal, the residual plot shows no pattern. No influential points detected. Assumptions are satisfied.”

22 Part 20 — Post-hoc Comparisons

22.1 20.1 When to use which

Comparison	Method
All pairwise, equal n	Tukey HSD
All pairwise, unequal n or variance	Games-Howell
Few comparisons	Bonferroni
Compare to control	Dunnett
Trend across ordered levels	Orthogonal polynomials
Simple effects in factorial	<code>emmeans(~A \ B)</code>

22.2 20.2 Tukey HSD

```
# --- Self-contained: Tukey HSD ---
set.seed(2001)
p20_df <- tibble(
  group = factor(rep(c("A", "B", "C", "D"), each = 10)),
  y = 10 + c(0, 2, 4, 6)[rep(1:4, each = 10)] + rnorm(40, 0, 1.5)
)
p20_fit <- aov(y ~ group, data = p20_df)
TukeyHSD(p20_fit)
```



```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov.default(formula = y ~ group, data = p20_df)
##
## $group
##      diff      lwr      upr      p adj
## B-A 1.909731 0.4824621 3.337001 0.0049862
## C-A 3.772422 2.3451523 5.199691 0.0000001
## D-A 6.899648 5.4723786 8.326917 0.0000000
## C-B 1.862690 0.4354210 3.289959 0.0063454
## D-B 4.989917 3.5626473 6.417186 0.0000000
## D-C 3.127226 1.6999571 4.554496 0.0000055
```

22.3 20.3 Bonferroni

```
# --- Self-contained: Bonferroni ---
set.seed(2002)
p20_df <- tibble(
  group = factor(rep(c("A", "B", "C", "D"), each = 10)),
  y = 10 + c(0, 2, 4, 6)[rep(1:4, each = 10)] + rnorm(40, 0, 1.5)
)
pairwise.t.test(p20_df$y, p20_df$group, p.adjust.method = "bonferroni")

##
## Pairwise comparisons using t tests with pooled SD
##
## data: p20_df$y and p20_df$group
##
##      A      B      C
## B 0.27113 -      -
## C 0.00024 0.08046 -
## D 5.9e-07 0.00034 0.34501
##
## P value adjustment method: bonferroni
```

22.4 20.4 emmeans (recommended)

```
# --- Self-contained: emmeans with different adjustments ---
set.seed(2003)
p20_df <- tibble(
  group = factor(rep(c("A", "B", "C", "D"), each = 10)),
  y = 10 + c(0, 2, 4, 6)[rep(1:4, each = 10)] + rnorm(40, 0, 1.5)
)
p20_fit <- aov(y ~ group, data = p20_df)
p20_emm <- emmeans(p20_fit, ~ group)

pairs(p20_emm, adjust = "tukey")

## contrast estimate SE df t.ratio p.value
## A - B      -0.978 0.601 36  -1.627  0.3768
## A - C      -3.599 0.601 36  -5.986 <0.0001
## A - D      -6.157 0.601 36 -10.244 <0.0001
## B - C      -2.621 0.601 36  -4.360  0.0006
```

```
## B - D      -5.180 0.601 36  -8.617 <0.0001
## C - D      -2.559 0.601 36  -4.257  0.0008
##
## P value adjustment: tukey method for comparing a family of 4 estimates
```

```
pairs(p20_emm, adjust = "bonferroni")
```

```
## contrast estimate      SE df t.ratio p.value
## A - B      -0.978 0.601 36  -1.627  0.6749
## A - C      -3.599 0.601 36  -5.986 <0.0001
## A - D      -6.157 0.601 36 -10.244 <0.0001
## B - C      -2.621 0.601 36  -4.360  0.0006
## B - D      -5.180 0.601 36  -8.617 <0.0001
## C - D      -2.559 0.601 36  -4.257  0.0008
##
## P value adjustment: bonferroni method for 6 tests
```

```
pairs(p20_emm, adjust = "holm")
```

```
## contrast estimate      SE df t.ratio p.value
## A - B      -0.978 0.601 36  -1.627  0.1125
## A - C      -3.599 0.601 36  -5.986 <0.0001
## A - D      -6.157 0.601 36 -10.244 <0.0001
## B - C      -2.621 0.601 36  -4.360  0.0003
## B - D      -5.180 0.601 36  -8.617 <0.0001
## C - D      -2.559 0.601 36  -4.257  0.0003
##
## P value adjustment: holm method for 6 tests
```

```
pairs(p20_emm, adjust = "none")  # unadjusted (for planned contrasts)
```

```
## contrast estimate      SE df t.ratio p.value
## A - B      -0.978 0.601 36  -1.627  0.1125
## A - C      -3.599 0.601 36  -5.986 <0.0001
## A - D      -6.157 0.601 36 -10.244 <0.0001
## B - C      -2.621 0.601 36  -4.360  0.0001
## B - D      -5.180 0.601 36  -8.617 <0.0001
## C - D      -2.559 0.601 36  -4.257  0.0001
```

22.5 20.5 Games-Howell (unequal variances)

```
# --- Self-contained: Games-Howell ---
set.seed(2004)
p20_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 15)),
  y = c(rnorm(15, 10, 1), rnorm(15, 12, 3), rnorm(15, 15, 5))
)
rstatix::games_howell_test(p20_df, y ~ group)
```

```
## # A tibble: 3 x 8
##   .y.  group1 group2 estimate conf.low conf.high p.adj p.adj.signif
## * <chr> <chr> <chr>      <dbl>    <dbl>    <dbl> <dbl> <chr>
## 1 y    A      B          1.43   -0.448     3.31 0.152 ns
## 2 y    A      C          3.83   -0.424     8.09 0.0805 ns
## 3 y    B      C          2.40   -2.09     6.89 0.382 ns
```

22.6 20.6 Simple effects for interactions

```
# --- Self-contained: simple effects ---
set.seed(2005)
p20_df <- expand.grid(
  A = factor(c("a1", "a2")),
  B = factor(c("b1", "b2"))
) %>%
  slice(rep(1:n(), each = 20)) %>%
  mutate(y = 10 + 2 * (A == "a2") + (B == "b2") +
    3 * (A == "a2") * (B == "b2") + rnorm(n(), 0, 1))

p20_fit <- aov(y ~ A * B, data = p20_df)
p20_emm <- emmeans(p20_fit, ~ A | B)
pairs(p20_emm, adjust = "bonferroni")
```

```
## B = b1:
## contrast estimate SE df t.ratio p.value
## a1 - a2 -1.97 0.292 76 -6.745 <0.0001
##
## B = b2:
## contrast estimate SE df t.ratio p.value
## a1 - a2 -5.17 0.292 76 -17.747 <0.0001
```

22.7 20.7 Reporting template

“Tukey HSD shows that group D differs significantly from A (mean difference 6.1, 95% CI [4.8, 7.4], $p < 0.001$), from B (difference 4.2, 95% CI [2.9, 5.5], $p < 0.001$), and from C (difference 2.1, 95% CI [0.8, 3.4], $p = 0.001$).”

23 Part 21 — Non-parametric Alternatives

23.1 21.1 One-way: Kruskal-Wallis

```
# --- Self-contained: Kruskal-Wallis ---
set.seed(2101)
p21_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y = c(rexp(20, 1), rexp(20, 0.5), rexp(20, 0.3))
)
kruskal.test(y ~ group, data = p21_df)

##
## Kruskal-Wallis rank sum test
##
## data: y by group
## Kruskal-Wallis chi-squared = 18.635, df = 2, p-value = 8.985e-05
```

Post-hoc: Dunn’s test with Bonferroni.

```
# --- Self-contained: Dunn's test ---
set.seed(2102)
p21_df <- tibble(
```

```

group = factor(rep(c("A", "B", "C"), each = 20)),
y      = c(rexp(20, 1), rexp(20, 0.5), rexp(20, 0.3))
)
rstatix::dunn_test(p21_df, y ~ group, p.adjust.method = "bonferroni")

## # A tibble: 3 x 9
##   .y. group1 group2    n1    n2 statistic      p    p.adj p.adj.signif
## * <chr> <chr> <chr> <int> <int>    <dbl>    <dbl>    <dbl> <chr>
## 1 y     A     B      20    20     2.33 0.0200  0.0599    ns
## 2 y     A     C      20    20     3.59 0.000325 0.000976 ***
## 3 y     B     C      20    20     1.27 0.205   0.615     ns

```

23.2 21.2 Repeated measures: Friedman

```

# --- Self-contained: Friedman test ---
set.seed(2103)
p21_df <- expand_grid(
  subject = factor(1:10),
  time    = factor(c("t1", "t2", "t3"))
) %>%
  mutate(y = rexp(30, 1 / (2 + as.numeric(time))))

friedman.test(y ~ time | subject, data = p21_df)

##
## Friedman rank sum test
##
## data: y and time and subject
## Friedman chi-squared = 6.2, df = 2, p-value = 0.04505

```

Post-hoc: Wilcoxon signed-rank with Bonferroni.

```

# --- Self-contained: Wilcoxon post-hoc ---
set.seed(2104)
p21_df <- expand_grid(
  subject = factor(1:10),
  time    = factor(c("t1", "t2", "t3"))
) %>%
  mutate(y = rexp(30, 1 / (2 + as.numeric(time))))

pairwise.wilcox.test(p21_df$y, p21_df$time,
  paired = TRUE,
  p.adjust.method = "bonferroni")

##
## Pairwise comparisons using Wilcoxon signed rank exact test
##
## data: p21_df$y and p21_df$time
##
##      t1      t2
## t2 1.00 -
## t3 0.58 0.58
##
## P value adjustment method: bonferroni

```

23.3 21.3 Factorial non-parametric: ART

```
# --- Self-contained: aligned rank transform ---
# library(ARTool)
# set.seed(2105)
# p21_df <- tibble(
#   A = factor(rep(c("a1", "a2"), each = 20)),
#   B = factor(rep(rep(c("b1", "b2"), each = 10), 2)),
#   y = rexp(40, 1)
# )
# art_fit <- art(y ~ A * B, data = p21_df)
# anova(art_fit)
```

23.4 21.4 Non-parametric summary

Parametric	Non-parametric alternative
One-way ANOVA	Kruskal-Wallis
Two-way ANOVA	ART / Friedman for RM
RM ANOVA	Friedman
Paired t-test	Wilcoxon signed-rank
Independent t-test	Mann-Whitney
Post-hoc Tukey	Dunn
Post-hoc paired	Wilcoxon with correction

23.5 21.5 Reporting template

“Because residuals were non-normal (Shapiro-Wilk $p = 0.002$), Kruskal-Wallis was used instead of ANOVA. The test was significant, $\chi^2(2) = 15.4$, $p < 0.001$. Post-hoc Dunn’s tests with Bonferroni correction show groups A and C differ significantly ($p = 0.002$).”

24 Part 22 — Theory Q&A

Q: What are the three pillars of experimental design? A: Randomization, replication, and blocking. Randomization eliminates bias, replication estimates error variance, and blocking reduces nuisance variation.

Q: Why randomize? A: To eliminate systematic bias, ensure independence, and make the F-test valid. Randomization is what allows causal inference.

Q: What is the difference between an experimental unit and an observational unit? A: An experimental unit is the smallest unit to which a treatment is applied independently. An observational unit is the unit on which the response is measured. They can be the same or different (e.g., measuring multiple plants in the same pot — the pot is the experimental unit).

Q: What is pseudo-replication? A: Treating subsamples as independent replicates when they are actually nested within the same experimental unit. Inflates Type I error. Fix: average subsamples or use a mixed model.

Q: What is confounding? A: When the effect of one factor is mixed with another. Example: if all treatment A units are in block 1 and all treatment B in block 2, treatment is confounded with block.

Q: What is the difference between a CRD and an RCBD? A: CRD has no blocking; RCBD blocks on a nuisance factor. RCBD requires that each treatment appear once per block (complete block). Both test the same treatment effect, but RCBD has higher power if the block effect is large.

Q: When would you use a Latin Square? A: When you have two nuisance factors that are both likely to affect the response, and when the number of levels of each nuisance factor equals the number of treatments.

Q: What is a factorial design? A: A design in which two or more factors are crossed, so that all combinations of levels are tested. The main advantage is the ability to detect interactions.

Q: What is an interaction? A: When the effect of one factor depends on the level of another. Significant interaction means main effects cannot be interpreted in isolation.

Q: What is a fractional factorial? A: A subset of the full factorial runs, used when full factorial is impractical. Some effects are aliased with others. The resolution describes which effects are aliased.

Q: What is the resolution of a fractional factorial? A: Resolution III: main effects aliased with 2-way interactions. Resolution IV: main effects aliased with 3-way interactions. Resolution V: main effects aliased with 4-way interactions. Higher resolution is better.

Q: What is a split-plot design? A: A design with two sizes of experimental units: whole plots and split plots. Used when some factors are hard to change (whole-plot) and others are easy to change (split-plot). Has two error terms.

Q: Why does split-plot need two error terms? A: Because the whole-plot factor is applied to larger units (more error variance) and the split-plot factor to smaller units (less error variance). Using one error term for both would give wrong F-tests.

Q: What is a nested design? A: A design in which the levels of one factor are unique within the levels of another. Example: classes nested within schools. Analysis uses a nested error term.

Q: What's the difference between crossed and nested? A: Crossed: every level of A is tested with every level of B. Nested: B is unique within A. Crossed designs can estimate $A \times B$ interactions; nested designs cannot.

Q: What is a crossover design? A: A design in which each subject receives multiple treatments in sequence. Used in clinical trials. Key risk: carryover effects (need washout).

Q: What is a fixed effect? A: An effect where the levels are chosen specifically and inference is only to those levels.

Q: What is a random effect? A: An effect where the levels are a random sample from a larger population and inference extends to that population.

Q: What is a mixed model? A: A model with both fixed and random effects. Used for repeated measures, hierarchical data, and complex designs.

Q: What is EMS and why does it matter? A: Expected Mean Square. It tells you which error term to use as the F-test denominator. Choosing the wrong denominator gives wrong p-values.

Q: When do you use a random effect for block? A: When the blocks are a sample from a larger population and you want to generalize. Use fixed block effects if blocks are specifically chosen.

Q: What is power and how does it relate to sample size? A: Power is the probability of detecting a true effect. It increases with sample size, effect size, and α . Standard target: 80% power.

Q: What is blocking efficiency? A: The ratio of error MS in a CRD to error MS in the corresponding RCBD. If $RE > 1$, RCBD is more efficient. Rule of thumb: block if the nuisance factor explains $> 20\%$ of variance.

Q: What is the difference between a split-plot and a repeated measures design? A: Split-plot has two sizes of experimental units; repeated measures has the same subjects measured multiple times. Both have within-subject factors, but split-plot is about hard-to-change factors while RM is about time or conditions.

Q: When should you use Type III SS? A: When the design is unbalanced (unequal cell sizes). Type III tests each effect after all others. Use `car::Anova(fit, type = "III")`.

Q: When should you use a non-parametric alternative? A: When residuals are non-normal (and n is small) or when the response is ordinal. Use Kruskal-Wallis for independent groups, Friedman for repeated measures.

25 Part 23 — Exam Scenarios

25.1 23.1 You are given a table of data and asked to analyze

1. **Read** the data (`read_excel`, `read_csv`).
2. **Identify** the design (look at the layout).
3. **Convert** types: factors with explicit reference.
4. **Fit** with the appropriate model.
5. **Check** assumptions.
6. **Post-hoc** if significant.
7. **Report** in words.

25.2 23.2 “Design an experiment to test X”

1. **State** the treatment factor and levels.
2. **State** the experimental unit.
3. **Choose** the design (CRD, RCBD, factorial, ...).
4. **Randomize** the assignment.
5. **Replicate** enough for power.
6. **Analyze** with the corresponding model.

25.3 23.3 “Test if factor A affects the response”

1. Fit `aov(y ~ A)` (or with covariates / blocks).
2. Read the F-test for A.
3. Report F, df, p.
4. Post-hoc if significant.
5. Report effect size.

25.4 23.4 “Compare 4 groups”

1. `aov(y ~ group)`.
2. If F significant, TukeyHSD or `emmeans`.
3. Report pairwise differences with CIs.

25.5 23.5 “Test interaction between A and B”

1. Fit `aov(y ~ A * B)`.
2. Look at the A:B row.
3. If significant, do simple effects: `emmeans(~ A | B)`.
4. Plot interaction.

25.6 23.6 “Analyze a repeated measures experiment”

1. Convert wide $\rightarrow$ long.
2. Fit `aov(y ~ time + Error(subject/time))`.
3. Read the F-test in the `subject:time` stratum.
4. Check sphericity (afex).
5. Post-hoc with paired comparisons.

25.7 23.7 “Analyze a split-plot experiment”

1. Identify whole-plot and split-plot factors.
2. Fit `aov(y ~ whole * split + Error(block / whole))`.
3. Report whole-plot F-test in the `block:whole` stratum.
4. Report split-plot F-tests in the `Within` stratum.

25.8 23.8 “Design and analyze a factorial experiment”

1. Identify factors and levels.
2. Choose a full or fractional factorial design.
3. Randomize run order.
4. Fit `aov(y ~ A * B * C)`.
5. Report main effects and interactions.
6. Plot interaction.

25.9 23.9 “Blocking vs no blocking”

Compute relative efficiency (Part 18). If $RE > 1$, blocking was helpful.

25.10 23.10 “Choose between two designs”

Compare: - Precision (RCBD $>$ CRD if block effect exists). - df for error. - Feasibility (block capacity). - Power.

26 Part 24 — Quick Reference

Task	Command
One-way ANOVA	<code>aov(y ~ g, data = df)</code>
Two-way ANOVA	<code>aov(y ~ A * B, data = df)</code>
RCBD	<code>aov(y ~ trt + block, data = df)</code>
Latin Square	<code>aov(y ~ row + col + trt, data = df)</code>
BIBD	<code>agricolae::BIB.test(block, trt, y)</code>
Factorial with blocks	<code>aov(y ~ block + A * B, data = df)</code>
Split-plot	<code>aov(y ~ whole * split + Error(block/whole), data = df)</code>
RM ANOVA (1 factor)	<code>aov(y ~ time + Error(subject/time), data = df)</code>
RM ANOVA (2 factors)	<code>aov(y ~ A * B + Error(subject/(A*B)), data = df)</code>
Mixed design	<code>aov(y ~ between * within + Error(subject/within), data = df)</code>
Mixed model	<code>lmer(y ~ A * B + (1 subject), data = df)</code>
ANCOVA	<code>lm(y ~ g + x, data = df)</code>
Type III SS	<code>car::Anova(fit, type = "III")</code>
Welch ANOVA	<code>oneway.test(y ~ g, data = df, var.equal = FALSE)</code>
Post-hoc Tukey	<code>TukeyHSD(fit)</code>
Post-hoc Bonferroni	<code>pairwise.t.test(y, g, p.adjust.method = "bonferroni")</code>
Post-hoc Games-Howell	<code>rstatix::games_howell_test(df, y ~ g)</code>

Task	Command
Adjusted means	<code>emmeans(fit, ~ group)</code>
Simple effects	<code>emmeans(fit, ~ A B)</code>
Trend test	<code>contrasts(df\$g) <- contr.poly(k)</code>
Normality	<code>shapiro.test(residuals(fit))</code>
Equal variances	<code>car::leveneTest(y ~ g, data = df)</code>
Kruskal-Wallis	<code>kruskal.test(y ~ g, data = df)</code>
Friedman	<code>friedman.test(y ~ t s, data = df)</code>
Effect size η^2_p	<code>effectsize::eta_squared(fit)</code>
Effect size η^2	<code>effectsize::omega_squared(fit)</code>
Power	<code>pwr.anova.test(k, n, f, sig.level, power)</code>
Design generation	<code>agricolae::design.crd(), design.rcbd(), design.lsd(), design.bib()</code>
Fractional factorial	<code>FrF2::FrF2(nruns, nfactors)</code>
Blocking efficiency	manual from ANOVA table
Sphericity correction	<code>afex::aov_ez(...)</code>

27 Part 25 — Common Mistakes

1. **Not randomizing** — biases results; invalidates inference.
2. **Confusing experimental unit with observational unit** — leads to pseudo-replication.
3. **Forgetting to block** when a nuisance factor is known and important.
4. **Blocking on the treatment factor** — blocks must be independent of treatments.
5. **Using one error term for split-plot** — must use two error terms.
6. **Ignoring the aliasing structure** in fractional factorials — some effects are confounded by design.
7. **Using aov without Error()** for repeated measures — treats within as between.
8. **Reading the top stratum in RM output** — the F-test is in the within-subject stratum.
9. **Forgetting to convert wide to long** before RM ANOVA.
10. **Ignoring sphericity violation** — inflates Type I error for within factors.
11. **Using Type I SS on unbalanced data** — use Type III.
12. **Not checking homogeneity of slopes** in ANCOVA.
13. **Interpreting main effects when interaction is significant** — use simple effects.
14. **Using TukeyHSD for unequal variances** — use Games-Howell.
15. **Not adjusting for multiple comparisons** — inflates Type I error.
16. **Reporting only p-values without effect size** — p depends on n, effect size does not.
17. **Using post-hoc on a non-significant omnibus test** — technically allowed but interpret with caution.
18. **Ignoring the EMS** — using the wrong denominator gives wrong p-values.
19. **Forgetting set.seed()** — random assignment is not reproducible.
20. **Treating a random effect as fixed** — wrong inference and wrong F-test.
21. **Not checking lmer convergence** — `isSingular()` and warnings matter.
22. **Using (1 + factor | subject) with few observations per subject** — unidentifiable model.
23. **Pseudo-replicating subsamples** — inflates Type I error.
24. **Post-hoc blocking after data collection** — weaker than pre-planned blocking.
25. **Not reporting degrees of freedom** — F-statistics alone are incomplete.

28 Part 26 — The Single Most Important Idea

Experimental design is about controlling variation.

- **Randomize** to eliminate bias.
- **Replicate** to estimate error.
- **Block** to reduce nuisance variation.
- **Choose the design** that matches the structure of the problem.
- **Choose the model** that matches the design.
- **Use the right error term** (from EMS) for each F-test.
- **Report** effects, sizes, and conclusions.

Every design question reduces to:

1. **What is the design?**
2. **What are the fixed and random effects?**
3. **What is the correct error term for each effect?**
4. **Is the effect significant?**
5. **How large is it?**
6. **What does it mean in practice?**

If you can answer these six questions, you can design, analyze, and interpret any experiment.

End of cheat sheet. Good luck in the competition. ““
